

M5. Création de classes



Ce module se concentre sur les aspects avancés du langage de programmation Python 3. Il explore en détail la **définition et la structure des classes**, l'**utilisation des constructeurs et destructeurs de classes**, ainsi que les **spécificités des attributs et des méthodes de classes et d'instances**. C'est un outil idéal pour ceux qui cherchent à approfondir leur connaissance en programmation orientée objet et à maîtriser les concepts avancés de Python.



Définition et structure des classes



Constructeur et destructeur



Attributs de classe et attributs d'instance



Méthode de classe et méthode d'instance



Review

Définition et structure des classes

En Python, les classes sont des structures de données qui regroupent des attributs (variables) et des méthodes (fonctions).

Elles sont utilisées pour créer des objets qui sont des instances spécifiques de ces classes.

La **définition d'une classe** comprend la **déclaration de ses attributs et méthodes**, ainsi que la **définition de son comportement**.

Pour pouvoir définir une classe correctement, il est nécessaire de connaître et d'utiliser la syntaxe appropriée :

- Le mot-clé **class** est utilisé pour déclarer une classe. On le fait suivre du nom de la classe à créer. Par convention, **le nom d'une classe commence toujours par une majuscule**.
- Le paramètre **self** permet de **faire référence à l'instance courante**, c'est-à-dire l'objet en cours de manipulation.

La création d'une classe répond au modèle de code suivant :

```
class NomDeClasse:  
    nomAttributDeClasse = Valeur de l'attribut
```

```
def methodeDeClasse(self, parametre1, parametre2, ...,parametreX.):  
    #traitement à effectuer
```

Structure de base de création d'une classe.

Une fois la classe définie, nous pouvons créer des objets à partir de cette classe en utilisant le modèle de code suivant :

```
monObjet = NomDeClasse(valeurParam1, valeurParam2, ..., valeurParamX)
```

Structure de base de création d'un objet.

Exemple d'une classe qui a pour objectif d'afficher des messages de logs :

```
class Message():  
  
    def afficherMessage(self, message):  
        print(message)
```

Script d'une classe qui affiche des messages.

Cette classe comporte une **méthode** **afficherMessage** en charge de l'affichage du message en console.

Voici comment utiliser cette méthode :

```
monObjMessage = Message()  
monObjMessage.afficherMessage("Bonjour le monde!")
```

Exemple de script qui utilise une méthode.

Premièrement, nous créons un objet message sur base de la classe Message. Ensuite, au moyen de l'**opérateur .** nous allons rechercher la méthode **afficherMessage** à laquelle nous fournissons le message attendu en paramètre.

Le résultat de ce script est:

```
Bonjour le monde!
```

Résultat du script précédent.

CONTINUER

Constructeur et destructeur

Constructeur

Lorsque nous créons une instance d'une classe afin d'obtenir un objet, nous participons à **la construction d'un objet**.

Un constructeur est appelé de manière automatique. C'est ce constructeur qui permet **l'instanciation**.

En l'absence de constructeur explicite au sein d'une classe que nousinstancions, un constructeur par défaut appartenant à la *superclass Object* est utilisé.

Ce constructeur est appelé **constructeur par défaut**.

Au delà de l'instanciation, **le rôle principal du constructeur est d'initialiser les attributs de l'objet**. Il est donc logique que ce constructeur apparaisse dans chaque classe que vous définirez.

Un constructeur n'est que la définition au sein de votre classe de la méthode spéciale **`__init__()`**.

Cette méthode doit prendre comme premier paramètre la paramètre self qui, comme expliqué dans la leçon précédente, est la référence à l'instance courante de la

classe.

C'est grâce à ce paramètre **self** que nous pouvons définir et modifier les attributs de l'objet.

Reprenons l'exemple de la création d'une classe Personne :

```
class Personne:
    def __init__(self):
        print("Création d'un objet Personne")

obj = Personne()
```

Script de création d'une classe Personne.

Ce code illustre le fonctionnement du constructeur. L'exécution nous renverra le résultat suivant en console :

```
Création d'un objet Personne
```

Résultat du script précédent.

L'intérêt lors de la construction d'un objet est de pouvoir préciser directement tout ou partie de ses attributs. Adaptons le constructeur afin qu'il puisse recevoir et définir les attributs *nom* et *prénom* :

```
class Personne:
    def __init__(self, param_nom, param_prenom):
        self.nom = param_nom
        self.prenom = param_prenom
```

```
        print(f"Création d'un objet Personne avec le nom '{self.nom}' et le  
obj = Personne("Moore", "Roger")
```

Script qui instancie un objet en lui fournissant des attributs.

Avec ce code, nous créons un objet *Personne* en passant le nom et le prénom en paramètre au constructeur de l'objet.

```
Création d'un objet Personne avec le nom Moore et le prénom Roger
```

Résultat du script précédent.

Lors de la construction de l'objet, nous définissons le nom et le prénom des attributs de la classe qui peuvent être utilisés dans notre code au moyen de l'opérateur `.` en suivant le modèle de code habituel :

```
En lecture : monObjet.monAttribut  
En écriture : monObjet.monAttribut = valeur
```

Destructeur

De la même manière qu'il existe un constructeur pour la création des objets, il existe également un destructeur pour leur destruction.

Un destructeur peut être utilisé pour s'assurer que les ressources utilisées par l'objet sont libérées telles qu'un fichier ouvert, une connexion à une base de données, etc.

Hormis ces cas spécifiques, il n'est pas particulièrement utile de définir notre propre destructeur au sein de nos propres classes et ce notamment en raison du cycle de vie des objets.

Les objets sont **gérés en mémoire** tout au long de leur utilisation au sein de l'application. Lorsque ceux-ci ne sont plus directement à la portée du code courant, on va dire de nos objets qu'ils ne sont plus gérés. C'est à partir de ce moment là qu'un processus de nettoyage de la mémoire appelé le **garbage collector** est susceptible de venir les nettoyer et donc les supprimer de la mémoire. C'est à ce moment que le destructeur sera utilisé.

Attention : nous ne contrôlons pas le *garbage collector* et donc le moment exact où le destructeur sera appelé.

Si, pour une raison pertinente de votre code, il est nécessaire de mettre en place un destructeur, il suffira de définir au sein de votre classe la méthode spéciale **`__del__()`** assortie du paramètre **`self`**. Au sein de cette méthode **`__del__()`**, vous pourrez exécuter tout le code qu'il vous semble pertinent pour libérer les ressources utilisées par votre objet.

```
class Personne:
    def __init__(self, param_nom, param_prenom):
        self.nom = param_nom
        self.prenom = param_prenom
        print(f"Création d'un objet Personne avec le nom '{self.nom}' et le prenom '{self.prenom}'")

    def __del__(self):
        print("Destruction de l'objet Personne")

obj = Personne("Moore", "Roger")
print("Fin du script. Notre destructeur sera appelé. Le programme s'arrête")
```

Script qui définit une classe comportant un destructeur (`__del__`).

L'exécution du script produira le résultat suivant :

```
Création d'un objet Personne avec le nom 'Moore' et le prenom 'Roger'
Fin du script. Notre destructeur sera appelé. Le programme s'arrête
Destruction de l'objet Personne
```

Résultat du script précédent.

Remarquez qu'à la fin du script, le destructeur de notre classe *Personne* sera appelé automatiquement.

Pour résumer les propos de cette leçon: le **constructeur `__init__()`** est utilisé pour initialiser les attributs d'un objet lors de sa création, tandis que le **destructeur `__del__()`** est utilisé pour libérer les ressources associées à l'objet lorsqu'il est détruit.

CONTINUER

Attributs de classe et attributs d'instance

Nous avons vu que les attributs qu'on retrouvait au sein d'une classe étaient, d'une certaine manière, les variables dédiées à cette classe.

Il faut distinguer deux types d'attributs différents : **les attributs de classe** et **les attributs d'instance**.

Les attributs de classe

Les attributs de classe sont des attributs qui sont définis au sein d'une classe **mais à l'extérieur de toute méthode de la classe**.

Ils sont initialisés lors de leur déclaration.

Les attributs de classes sont des attributs qui **ne dépendent pas d'une instance** particulière de la classe. Aussi, **leur valeur sera partagée par toutes les instances de la classe**.

Ces attributs n'étant pas dépendant d'une instance, ils sont dès lors accessibles par simple utilisation de la classe (et non d'un objet).

Voyez plutôt l'exemple suivant :

```
class Personne:
    denomination = "personne"

    def __init__(self):
        pass

objet1 = Personne()

print(Personne.denomination)
print(objet1.denomination)
```

Script définissant une classe qui comporte un attribut de classe.

 Le mot-clé **pass** permet de terminer une structure de code incomplète sans générer d'erreur.

Dans mon cas, un constructeur DOIT impérativement contenir du code mais **pass** me permet de **contourner cette contrainte** sans lever d'exception dans mon code.

Nous avons créé une classe *Personne* qui dispose d'un attribut de classe *denomination*.

Une instance de la classe est créée (*objet1*).

Nous affichons l'attribut de classe depuis l'utilisation de la classe et nous faisons la même chose depuis l'utilisation de l'objet.

Ce qui donne :

```
personne  
personne
```

Résultat du script précédent.

1/ Que se passe-t-il si nous changeons la valeur de notre attribut de classe entre les deux affichages ?

```
class Personne:  
    denomination = "personne"  
  
    def __init__(self):  
        pass  
  
objet1 = Personne()  
  
print(Personne.denomination)  
Personne.denomination = "humain"  
print(objet1.denomination)
```

Script qui modifie la valeur d'un attribut de classe.

Nous remarquons qu'après changement de la valeur de notre attribut de classe, notre deuxième affichage sera également impacté.

```
personne
```

```
humain
```

Résultat du script précédent.

2/ Que se passe-t-il si nous changeons la valeur de notre attribut de classe depuis une instance de celle-ci ?

```
class Personne:
    denomination = "personne"

    def __init__(self):
        pass

objet1 = Personne()
objet1.denomination = "humain"

print(objet1.denomination)
print(Personne.denomination)
```

Script qui modifie la valeur d'un attribut d'une instance.

Le résultat de notre script différera du premier exemple :

```
humain
personne
```

Résultat du script précédent.

La modification ayant été effectuée sur une instance de la classe, l'attribut de classe n'a pas été modifié. Le langage a considéré que nous faisons une modification d'attribut d'instance c'est-à-dire d'un attribut attaché à un objet spécifique.

Les attributs d'instance

Les **attributs d'instance** sont des attributs déclarés au sein d'une classe ET définis à l'intérieur de constructeur de la classe.

Ces attributs **ne sont accessibles qu'à partir d'une instance de la classe** (à partir d'un objet) et seront utilisés au sein de la classe au moyen du mot-clé **self**.

Ils sont utilisés pour stocker des données spécifiques à chaque instance.

Si nous reprenons notre classe `Personne` et que nous lui ajoutons des attributs d'instance, nous pourrions par exemple spécifier le nom et le prénom :

```
class Personne:
    def __init__(self, prenom, nom):
        self.prenom = prenom
        self.nom = nom

obj = Personne("John", "Doe")
print(obj.prenom)
print(obj.nom)
print(Personne.prenom)
```

Script qui déclare des attributs d'instance.

Les attributs d'instances seront initialisés grâce aux paramètres du constructeur.

Dans cet exemple, nous avons créé l'objet, affiché le nom et le prénom de la personne créée.

Que se passe-t-il si nous cherchons à utiliser l'attribut d'instance comme un attribut de classe ? Voici la réponse suite à l'exécution du script :

```
John  
Doe  
AttributeError: type object 'Personne' has no attribute 'prenom'
```

Résultat du script précédent.

Vous constatez qu'il n'est **PAS POSSIBLE** d'utiliser un attribut d'instance comme attribut de classe, contrairement à l'attribut de classe qui peut être utilisé comme attribut d'instance bien que je vous décourage fortement de le faire.

En résumé, les attributs de classe sont partagés par toutes les instances d'une classe, tandis que les attributs d'instance sont spécifiques à chaque instance. Les attributs de classe sont utilisés pour stocker des données communes à toutes les instances, tandis que les attributs d'instance sont utilisés pour stocker des données spécifiques à chaque instance.

CONTINUER

Méthode de classe et méthode d'instance

De la même manière que nous retrouvons des attributs de classe et des attributs d'instance, nous pouvons aussi retrouver des **méthodes de classe** et des **méthodes d'instance**.

Méthode de classe

Une méthode de classe est une méthode qui sera associée à la classe plutôt qu'à une instance spécifique de la classe.

Ce type de méthode permet de manipuler des attributs de classe mais n'a bien entendu aucun accès aux attributs d'instance.

Pour définir une méthode de classe, nous utiliserons l'annotation **@classmethod**.

Une méthode de classe prendra toujours en premier paramètre conventionnel le paramètre **cls** (pour classe). Ce paramètre permettra lors de l'utilisation de la méthode de classe, de pouvoir créer une instance de l'objet.

Voyez plutôt l'exemple suivant qui se base sur notre classe `Personne` :

```
class Personne:
    def __init__(self, prenom, nom):
        self.prenom = prenom
        self.nom = nom

    @classmethod
    def creerObjet(cls, prenom, nom):
        return cls(prenom, nom)

obj = Personne.creerObjet("Jean", "Claude")
print(obj.prenom)
```

Script qui utilise une méthode de classe.

La méthode de classe **creerObjet** est créée. À la fin de l'exécution de cette méthode, nous retournons un nouvel objet Personne en utilisant le constructeur tel que nous l'avons défini (avec 2 paramètres).

Le résultat de l'exécution de ce script sera le suivant :

```
Jean
```

Résultat du script précédent.

Nous obtenons donc bel et bien un nouvel objet dont les attributs ont été initialisés par appel du constructeur `__init__()`.

Méthode d'instance

Les **méthodes d'instance** sont des méthodes propres à chaque instance d'une classe.

Elles sont définies au sein de la classe et prennent un premier paramètre **self** représentant l'objet courant et permettant l'interaction avec les membres (attributs et méthodes) d'instance de la classe.

Si nous reprenons notre classe `Personne` et que nous lui ajoutons une méthode d'instance `sePresenter`, nous pourrions communiquer avec les attributs d'instance et donc personnaliser le message que nous souhaitons afficher lors de l'appel de la méthode :

```
class Personne:
    def __init__(self, prenom, nom):
        self.prenom = prenom
        self.nom = nom

    @classmethod
    def creerObjet(cls, prenom, nom):
        return cls(prenom, nom)

    def sePresenter(self):
        return f"Bonjour, je m'appelle {self.prenom} {self.nom}"

obj = Personne.creerObjet("Jean", "Claude")
print(obj.sePresenter())
```

Script qui utilise une méthode d'instance.

Comme les attributs d'instance n'étaient pas accessibles lors de l'utilisation de la classe, les méthodes d'instance ne sont également pas accessibles.

En résumé, les méthodes de classe sont associées à la classe elle-même et peuvent être appelées directement sur la classe, tandis que les méthodes d'instance sont spécifiques à chaque instance de la classe et sont appelées sur une instance spécifique. Les méthodes de classe sont généralement utilisées pour effectuer des opérations liées à la classe, tandis que les méthodes d'instance sont utilisées pour effectuer des opérations liées à une instance.

CONTINUER

Review

De quoi avons nous besoin pour créer une classe ?

☐

Du mot clé **class** utilisé lors de la déclaration

☐

Du mot clé **this** utilisé pour faire référence à l'instance courante de ma classe

☐

D'un nom à donner à la classe. Ce nom commence conventionnellement par une majuscule.

☐

D'attributs et de méthodes

SUBMIT

Écrivez une classe *Participant* permettant de modéliser le participant d'une formation FAD.

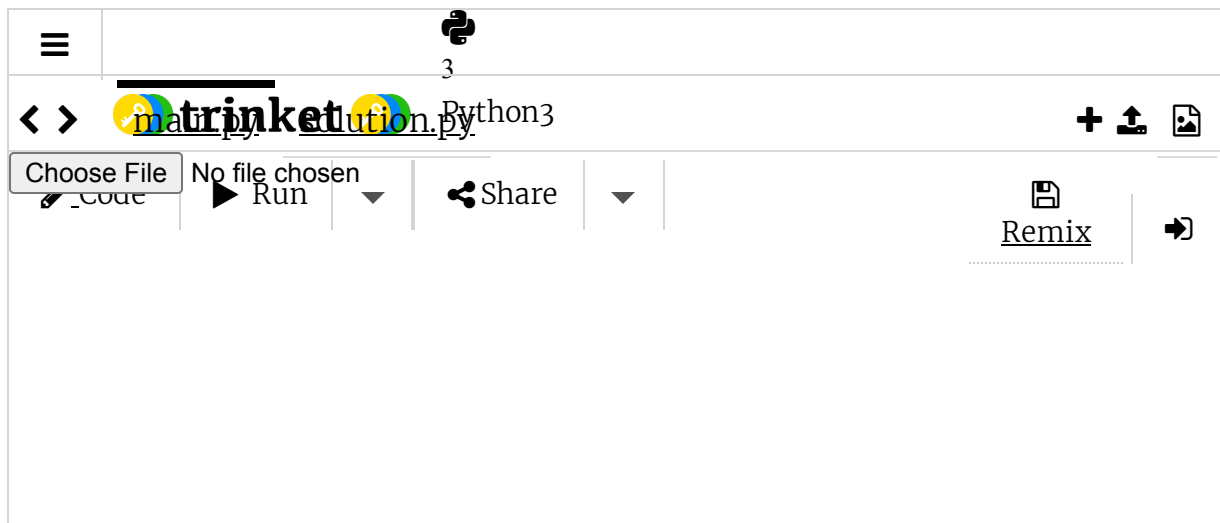
Un participant est caractérisé par un nom, un prénom ainsi qu'une liste de formations qu'il suit.
Il faut pouvoir :

1. Afficher les caractéristiques d'un participant :

" Le participant s'appelle [prenom] [nom], il suit actuellement [nombre de formations] formation(s). Voici les formations suivies: [formations]"

2. Ajouter une formation à la liste des formations que suit le participant.

Le participant sera, au minimum, construit sur base de son nom et de son prénom.



Qu'est ce qui définit une méthode de classe?

- ☐ Une méthode de classe est une méthode qui sera associée directement à la classe dans laquelle elle est définie grâce à la notation `@classmethod`
- ☐ Une méthode de classe est une méthode propre à chaque instance d'une classe.

SUBMIT

CONTINUER